

Explainable Hybrid Deep Learning IDS for IoT Networks

Complete guide to the final, honest version of the project: what went wrong first, how we fixed it, the real results, where the code lives, and how to defend it. Every number here comes from docs/FINAL_RESULTS.md (the source of truth); if an older document disagrees, the older document is wrong.

1. Core Story & The 4 Pillars

© The project in one paragraph

Goal: an IDS for IoT that is **accurate** (catches attacks, few false alarms), **explainable** (shows why it alarmed) and **lightweight** (fits an IoT gateway). Proposed model: a hybrid **CNN-GRU** with **SHAP** explanations. Main dataset: **Edge-IIoTset** (2,219,201 packets, Normal + 14 attack types). Second dataset: **CICIoT2023** (flow-level, 33 attacks in 7 categories + Benign).

⚠️ What we first saw, and why it was fake

Every first model (Random Forest, CNN, GRU, CNN-GRU) scored **100%** on Normal vs Attack. We investigated instead of celebrating. Root cause: the dataset writes an *empty* protocol field as "0" in the Normal captures and as "0.0" in the attack captures. After one-hot encoding these become different columns, so one column alone gives away the label (empty `dns.qry.name.len`: "0" = 1,613,798 Normal rows and 0 attacks; "0.0" = 0 Normal rows and 603,331 attacks). The same happens in `mqtt.topic`, `mqtt.protoname`, `mqtt.conack.flags`. Our earlier "fix" of removing MQTT did not help, because the DNS column carries the same artifact, and our SHAP plots had hidden it by stripping the `_0` / `_0.0` suffix from feature names.

🔗 Honest pipeline flow (final)

Raw Edge-IIoTset → **Canonicalise** ("0" = "0.0") → **Strict feature policy** (drop 7 per-packet identifiers + all `mqtt.*`; 51 encoded features) → **Shortcut scans** (single-feature stumps, depth-3 tree) → **GPU training**, 3 seeds, threshold chosen on validation only → **Tuned CNN-GRU** (Macro-F1 0.858, 79,169 params) → **Detection ceiling audit** → **SHAP + fidelity test** → **Edge benchmark** (FP16, 159 KB) → **Repeat on CICIoT2023**.

🏛️ Pillar 1: Honest Accuracy

After removing the shortcut, the tuned CNN-GRU scores **Macro-F1 0.858** (90.0% accuracy, 0.7% false alarms, 65.3% of attacks detected). A measured ceiling shows even a perfect memoriser detects only **67.5%**, so the model is at the limit of what these packet features allow.

🏛️ Pillar 2: Proven Explainability

SHAP top features are real TCP behaviour (`tcp.flags`, `tcp.connection.rst`, `tcp.flags.ack`, `tcp.len`, `tcp.connection.fin`). Retraining without the top-5 SHAP features drops Macro-F1 **0.858 → 0.605**; without 5 random features it stays 0.858. SHAP is faithful.

Pillar 3: Edge Feasibility

Final CNN-GRU is **315 KB**; the FP16 version is **159 KB** with no loss (0.858). On 1 CPU thread: about 6.5 ms per packet. INT8 on the GRU shrinks it to 88 KB but costs 7.6 points.

Pillar 4: Second Dataset

On **CICIoT2023** (flow-level, has rate and timing) we found and removed a **second hidden shortcut** (window size). Honest binary Macro-F1: CNN-GRU **0.890**, XGBoost **0.917**; 8 categories: 0.675 / 0.752.

FIGURE (RESULTS/FIGURES/)

WHAT IT SHOWS

[fig1_fake_vs_honest.png](#)

Leaky 100% vs honest 0.858 / 0.868

[fig2_shap_fidelity.png](#)

Removing top-5 SHAP vs 5 random features

[fig3_model_comparison.png](#)

Ablation: MLP, 1D-CNN, GRU, CNN-GRU, XGBoost

[fig4_datasets.png](#)

Edge-IIoTset vs CICIoT2023

[fig5_detection_ceiling.png](#)

Detection ceiling (67.5%) vs our models

2. Code Walkthrough: What to Open & Show

If faculty asks to see the code, follow the story: evidence of the shortcut → the fix → training → explanations → experiments → results.

audit_shortcuts.py

Shortcut evidence: `--raw` streams the CSV and counts how each class writes empty fields (`results/audit/empty_token_audit.csv`). Also fits one-feature stumps and a depth-3 tree on processed data to catch any remaining giveaway.

 **Say:** "This is the proof that the 100% was fake: one column, written as "0" vs "0.0", separates Normal from Attack."

src/preprocessing.py

The fix: `canonicalize_column()` merges "0" and "0.0" before encoding; `feature_fingerprint()` writes a fingerprint of the processed data into `metadata.json`. Streaming, low-RAM (peak 6.1 → 3.4 GB).

 **Say:** "We canonicalise numeric text, apply the strict feature policy, and stamp the data with a fingerprint so no model can be trained or explained on the wrong data."

src/training.py

Training engine: data held on the GPU (≈ 8 s/epoch vs ≈ 31 s), Adam, early stopping on validation loss, optional sqrt class weights, and `choose_threshold()` which picks the decision threshold on *validation* only.

 **Say:** "The test set is never used for any choice. The threshold comes from validation, and every comparison is run with 3 seeds."

src/models.py

Architectures: MLP, CNN1D, GRUBaseline and the proposed CNN_GRU (Conv1D → BatchNorm → ReLU → MaxPool → Dropout → GRU → Dense → output).

 **Say:** "The convolution learns local feature combinations, the GRU reads the resulting feature map step by step. The same file holds the simpler baselines we compared against."

src/explainability.py

SHAP engine: shap.GradientExplainer.
_check_data_matches_experiment()
refuses to run if the data fingerprint
differs from the one the model was
trained on.

Say: "An earlier SHAP plot
explained a model with the wrong
inputs. This guard makes that
impossible now."

run_clean_baselines.py

Clean baselines: XGBoost (GPU) and
CNN-GRU on the canonicalised data, per
feature policy (e.g. --policy
strict_no_mqtt --stage binary).

Say: "These are the honest
numbers after the fix, for both a tree
model and our network."

run_tune_cnn_gru.py

Tuning: Optuna, 20 trials, then retrain
the best setting with 3 seeds
(F_strict_no_mqtt_binary_best_s42/s7/s202).

Say: "Best validation 0.8596, test
0.858, the same as untuned. Tuning
cannot beat the ceiling."

run_final_studies.py

Final studies: ablation (MLP / 1D-CNN /
GRU, 3 seeds), SHAP on the final model,
SHAP fidelity (drop top-5 vs 5 random
features), edge benchmark.

Say: "This is where we test our own
claims: is the hybrid needed, and is
SHAP faithful?"

 `run_edge_benchmark.py`

Edge test, CPU only: FP32, FP16, INT8 versions and XGBoost; Macro-F1, file size, single-packet latency on 1 thread, throughput →
results/edge_benchmark.csv.

 **Say:** "A gateway has no GPU, so we measured on one CPU thread."

 `audit_ceiling.py`

Detection ceiling: groups identical feature vectors; if a pattern is mostly Normal, any classifier must call it Normal →
results/audit/detection_ceiling.json.

 **Say:** "Only 11,122 distinct patterns in 1.55 million training packets; even a perfect memoriser detects 67.5% of attacks."

 `run_ciciot.py + src/ciciot.py`

Second dataset: same honest pipeline on CICIoT2023 (`--policy strict`): duplicates removed before splitting, `source_file`, `packet_count` and `total_sum` dropped, flag counts turned into fractions.

 **Say:** "We found a second shortcut here: the window size reveals the label. We removed it too."

 `watch_dashboard.py`

Live terminal dashboard: reads the status/progress files and the registry while a run is going; can be opened or closed without affecting the run.

 **Say:** "Long runs are monitored live: epoch, loss, early-stopping counter, finished experiments."

 `make_figures.py`

Figures: builds
results/figures/fig1..fig5 from the
committed result files.

 **Say:** "Every chart is generated from
result files, not drawn by hand."

Where the actual results live

Master registry: results/experiment_registry.csv (every experiment, including the old leaky E01–E08 ones; final Edge-IIoTset runs are N01/N02, F_, A_, FID_; final CICIOT2023 runs are CS01–CS05).

Final model: results/experiments/F_strict_no_mqtt_binary_best_s42/best_model.pt
(+ experiment_record.json, confusion matrix, SHAP plots).

Audits: results/audit/empty_token_audit.csv,
results/audit/detection_ceiling.json.

Other results: results/shap_fidelity.json, results/edge_benchmark.csv,
results/operating_points.csv.

Charts: results/figures/ (fig1 to fig5).

3. Deep Dive: What is `best\_model.pt`?

What does it represent?

best_model.pt is the PyTorch file holding the trained weights (the state_dict). The training loop saves it every time validation loss improves, so it is the best epoch on validation. The final one is

results/experiments/F_strict_no_mqtt_binary_best_s42/best_model.pt: the tuned CNN-GRU with **79,169 parameters, 315 KB** on disk (FP16 copy: **159 KB**). It is the file the SHAP script and the edge benchmark load.

Do not show the old checkpoints as final

Older folders such as E01-E08 were trained on the leaky data (before the "0"/"0.0" fix). Their "100%" results and their sizes belong to the "what we first saw" story only.

MODEL (EDGE-IIOTSET, FINAL SETTING)	MACRO-F1	PARAMETERS	FILE SIZE
MLP	0.858	5,441	24 KB
1D-CNN	0.858	25,857	106 KB
GRU	0.858	24,577	99 KB
CNN-GRU (tuned, ours), FP32	0.858	79,169	315 KB
CNN-GRU, FP16 weights	0.858	79,169	159 KB
CNN-GRU, INT8 (GRU + Linear)	0.782	79,169	88 KB
XGBoost (GPU-trained)	0.868	226 trees	492 KB

Honest reading: every network lands on the same 0.858, so on per-packet features the hybrid gives **no accuracy gain** over an MLP. All of them sit at the measured ceiling. See [results/figures/fig3_model_comparison.png](#).



4. IoT Real-World Deployment Architecture

Sensors (smart meters, bulbs, temperature probes) do not run the AI. The model is meant to run on the **IoT edge gateway** (e.g. a Raspberry-Pi-class device). We benchmarked on a laptop CPU with 1 thread; testing on a real Pi is a next step.

🔗 THE 5-STEP EXECUTION LIFECYCLE

- 1. Packet Ingestion:** Device traffic passes through the gateway.
- 2. Feature Extraction:** Gateway extracts the same 51 encoded features used in training (same canonicalisation and feature policy).
- 3. AI Inference:** FP16 CNN-GRU (159 KB) scores the packet, about 6.5 ms per packet on 1 CPU thread.
- 4. Decision:** Score above the validation-tuned threshold → **Attack alert** ; otherwise Normal. The threshold can be moved to trade detections against false alarms.
- 5. Explainable Alert:** SHAP shows which features pushed the score (e.g. unusual `tcp.flags` / `tcp.connection.rst`). A live demo through the dashboard predict API is a next step.

VERSION (1 CPU THREAD)	MACRO-F1	SIZE	LATENCY / PACKET	THROUGHPUT
CNN-GRU FP32	0.858	315 KB	6.4 ms	1,500 rows/s
CNN-GRU FP16 weights	0.858	159 KB	6.5 ms	1,660 rows/s
CNN-GRU INT8 (Linear only)	0.858	304 KB	8.1 ms	1,620 rows/s
CNN-GRU INT8 (GRU + Linear)	0.782	88 KB	13.3 ms	2,050 rows/s
XGBoost	0.868	492 KB	0.8 ms	30,700 rows/s

Honest reading: FP16 halves the size with no loss. XGBoost is faster on a CPU; the networks are smaller (MLP only 24 KB). With 65.3% of attacks detected on packet data, the system is an alerting tool, not an automatic packet-dropper.



5. Complete Parameter Guide

A. Training hyperparameters (final model, chosen by Optuna, 20 trials)

PARAMETER	VALUE	SIMPLE EXPLANATION
Batch Size	8192	Packets processed before each weight update (large, because the data sits on the GPU).
Learning Rate	0.0030	Step size of the Adam optimizer.
Optimizer	Adam	Adapts the step size per weight.
Dropout	0.239	Randomly switches off about 24% of neurons during training to prevent memorising.
Loss	Cross-entropy (binary)	Penalises confident wrong answers. Plain cross-entropy for the binary model; the loss study (sqrt weights vs focal vs cross-entropy) was run on the multiclass task.
Epochs	15	Passes over the training data, with early stopping on validation loss; best_model.pt keeps the best epoch.
Decision Threshold	Tuned on validation	The score cut-off for "Attack"; chosen on validation data only, never on test.
Seeds	3 (42, 7, 2024)	Same run repeated with different randomness: 0.8584 / 0.8583 / 0.8582.

B. Architecture (neural anatomy)

LAYER	SETTING	SIMPLE EXPLANATION
Conv1D	64 filters, kernel 5	64 small detectors, each looking at 5 neighbouring features at a time.
BatchNorm + ReLU + MaxPool(2)	-	Stabilises training, keeps positive signals, halves the length of the feature map.
GRU	hidden 128	Reads the pooled feature map step by step and keeps a 128-number memory.
Dense	32 units → 1 output	Combines the GRU summary into one attack score.
Total Learnable Params	79,169	All numbers stored in <code>best_model.pt</code> (315 KB FP32, 159 KB FP16).

C. Input features (Edge-IIoTset final setting: `strict_no_mqtt`, 51 encoded features)

✓ Kept

TCP: `tcp.flags`, `tcp.flags.ack`, `tcp.len`, `tcp.connection.syn/synack/fin/rst`.
HTTP: `method`, `version`, `referer`, `content length`, `response`. **DNS:** `query name/length/type`, `retransmissions`. Plus ARP, ICMP and Modbus length fields.

✗ Removed

Starting from the operational policy (already no IPs, ports or timestamps): minus 7 per-packet identifiers (`tcp.seq`, `tcp.ack`, `tcp.ack_raw`, `tcp.checksum`, `icmp.checksum`, `icmp.seq_le`, `udp.stream`) and minus every `mqtt.*` field (every MQTT packet in the dataset is Normal).

CICIoT2023 uses flow-level features (e.g. header_length, rate, flag fractions, protocol indicators) with source_file, packet_count and total_sum removed.



6. Results & Metric Cheat-Sheet

Metric	Our Score (Edge-IIOTSet, Binary, Tuned CNN-GRU)	Plain Language Meaning
Accuracy	90.0%	Share of all packets classified correctly. Misleading on imbalanced data (most packets are Normal), so we lead with Macro-F1.
Macro-F1	0.858 (XGBoost 0.868)	F1 of each class averaged equally, so the smaller Attack class counts as much as Normal. Our headline number.
Detection Rate (Recall)	65.3%	Of all real attacks, how many we caught.
False-Alarm Rate (FPR)	0.7%	Of all normal packets, how many we wrongly flagged.
Precision	High (few false alarms)	When the alarm goes off, how often it is a real attack.
ROC-AUC	0.904 (XGBoost 0.917)	How well scores rank attacks above normal packets across all thresholds; 1.0 = perfect, 0.5 = guessing.
Detection Ceiling	67.5% at 0.5% false alarms	Best any model can do on these features: 31.9% of attack packets look exactly like mostly-Normal patterns. We reach 65.3% (XGBoost 67.3%).
Operating Point	73.3% detection at 6.6% false alarms	Lowering the threshold catches more attacks but raises false alarms.

Metric	Our Score (Edge-IIoTSet, Binary, Tuned CNN-GRU)	Plain Language Meaning
SHAP Fidelity	Top-5 removed: 0.858 → 0.605; 5 random: 0.858 → 0.858	Proof that the features SHAP ranks highest are the ones the model really depends on.
Latency / Size	6.5 ms, 159 KB (FP16)	Time per packet on 1 CPU thread; size on disk.
Multiclass Macro-F1	0.337 ± 0.033 (XGBoost 0.429)	Attack <i>type</i> (14 + Normal) is weak on single packets: floods and scans need packet rate, which one packet cannot show.

CICIOT2023 (Flow-Level)	XGBoost	CNN-GRU
Benign vs Attack, Macro-F1	0.917	0.890
Detection / False alarms	93.4% / 8.4%	90.4% / 9.6%
ROC-AUC	0.979	0.966
Detection at ≤ 1% false alarms	86.6%	82.3%
8 categories, Macro-F1	0.752	0.675 (sqrt weights; cross-entropy 0.620)

Edge-IIoTset → CICIOT2023 (CNN-GRU): binary 0.86 → 0.89, categories 0.34 → 0.68, because flow data carries rate and timing. On CICIOT2023

SHAP ranks header_length, ack_count_frac, rate, https, arp highest.
Figures: fig1_fake_vs_honest.png, fig4_datasets.png,
fig5_detection_ceiling.png.

7. 5–Minute Faculty Presentation Script

STAGE 1: PROBLEM STATEMENT (40 SEC)

"Good morning sir/ma'am. Our project is **Explainable Hybrid Deep Learning-Based Intrusion Detection for IoT Networks**. IoT devices are easy to attack and too weak to run heavy security software. We want an IDS that is **accurate**, **explainable**, and **light enough for an IoT gateway**. Our model is a hybrid **CNN-GRU** with **SHAP** explanations, tested on **Edge-IIoTset** and **CICIoT2023**."

STAGE 2: THE 100% WAS FAKE (60 SEC)

"Our first models all scored **100%**. Instead of celebrating, we investigated. The dataset writes an empty field as **"0" in normal traffic and "0.0" in attack traffic**, so after encoding a single column gives away the label. We fixed it by treating "0" and "0.0" as the same value, removed 7 per-packet identifiers and all MQTT fields, and added automatic shortcut scans and data-fingerprint guards. Figure 1 shows the fake 100% next to the honest result."

STAGE 3: HONEST RESULTS & THE CEILING (70 SEC)

"On the clean data, our tuned CNN-GRU reaches **Macro-F1 0.858**: 90% accuracy, **0.7% false alarms**, and it detects **65.3% of attacks**. It is stable across 3 seeds, and tuning with 20 Optuna trials does not raise it. We measured why: the 1.55 million training packets have only 11,122 distinct feature patterns, and a third of attack packets look exactly like normal ones. Even a perfect memoriser detects only **67.5%**, so our model is **at the ceiling** of what single packets allow."

STAGE 4: EXPLAINABILITY THAT IS PROVEN (50 SEC)

"SHAP says the model relies on real TCP behaviour: **tcp.flags, connection resets, ACK flags, packet length** . We did not just trust the plot. We retrained without SHAP's top-5 features and Macro-F1 fell from **0.858 to 0.605** ; removing 5 random features changed nothing. So the explanation is faithful."

STAGE 5: EDGE & SECOND DATASET (50 SEC)

"The final model has 79,169 parameters; in FP16 it is **159 KB** with no loss, at about 6.5 ms per packet on one CPU thread. On **CICIoT2023** , which has flow-level rate and timing, we found and removed a second hidden shortcut, and the CNN-GRU reaches **0.890** binary and **0.675** on 8 attack categories, up from 0.34 on packet data."

STAGE 6: HONEST LIMITS & NEXT STEPS (40 SEC)

"To be transparent: on per-packet features the CNN-GRU is **not better than an MLP or XGBoost** , because the GRU sees no real time dimension. Next we will give the GRU **real sequences of flow windows** , run a cross-dataset test, and deploy the FP16 model on a Raspberry-Pi-class device. Our contribution: two hidden dataset shortcuts found and removed, a measured ceiling, proven SHAP fidelity, and a fully reproducible evaluation. Thank you."



8. Tough Faculty Defense Q&A

❓ **Q1: Your first models got 100%. Why is the final result lower?**

"The 100% was fake. Empty fields were written as "0" in normal captures and "0.0" in attack captures, so one column revealed the label. Every model, including Random Forest, exploited it. After the fix the honest result is **0.858 Macro-F1**. A lower honest number is worth more than a perfect fake one."

❓ **Q2: Isn't 65% detection too low for an IDS?**

"It is low, and we measured why: **31.9% of attack packets** have exactly the same features as mostly-normal patterns, so no packet-level model can catch them. The ceiling is **67.5%**; we reach 65.3% (XGBoost 67.3%). The threshold can be lowered to reach 73.3% at 6.6% false alarms. With flow-level data (CICIoT2023) detection rises to 90.4%."

❓ **Q3: If an MLP gets the same 0.858, why the hybrid CNN-GRU?**

"Honest answer: on per-packet features the hybrid gives **no accuracy gain**; MLP, 1D-CNN, GRU and CNN-GRU all score 0.858 because they all hit the ceiling. The GRU needs a real time dimension to help. Our next step is to feed it consecutive flow windows per device or connection, which is where a recurrent model should pay off."

❓ **Q4: XGBoost is slightly better and much faster. Why not just use it?**

"XGBoost is a strong baseline and we report it openly: 0.868 vs 0.858, 0.8 ms vs 6.5 ms per packet. The networks are smaller (FP16 CNN-GRU 159 KB, MLP 24 KB, XGBoost 492 KB). If CPU speed on per-packet data were the only goal, XGBoost would be a fair choice; the CNN-GRU is kept because the next step feeds it real sequences of flow windows, where a recurrent model can use time and a tree model cannot."

❓ **Q5: How do you know SHAP is telling the truth?**

"We tested it. Retraining without SHAP's **top-5 features** drops Macro-F1 from 0.858 to **0.605**; retraining without **5 random features** changes it by only 0.0001. We also added a guard so SHAP refuses to run on data the model was not trained on, because an earlier plot had explained a model with the wrong inputs."

❓ **Q6: Earlier you said MQTT fields drive detection. What changed?**

"That came from the leaky data: the MQTT fields carried the same "0"/"0.0" artifact, and every MQTT packet in the dataset is Normal. We now remove all MQTT fields; with vs without MQTT gives 0.8579 vs 0.8575, so the model does not use them. The real top features are TCP flags, resets, ACKs and packet length."

? **Q7: Why is attack-type (multiclass) detection so weak?**

"On Edge-IIoTset strict features, Macro-F1 is 0.337 for CNN-GRU and 0.429 for XGBoost. Floods and scans are defined by packet *rate*, which a single packet cannot show, and the dataset has no timing for attack packets (`udp.time_delta` is 0 for every attack row). On CICIoT2023, which has rate, it rises to 0.675 (CNN-GRU) and 0.752 (XGBoost)."

? **Q8: Did you try focal loss or class weights for imbalance?**

"Yes, over 3 seeds on multiclass: sqrt class weights **0.337** beat focal loss 0.254 and cross-entropy 0.238. Sqrt weights also help on CICIoT2023 categories (0.675 vs 0.620). For the binary model, plain cross-entropy was kept."

? **Q9: How can we trust there is no other leakage?**

"Automatic shortcut scans fit a one-feature stump per column and a depth-3 tree; a giveaway shows up as near-100%. On CICIoT2023 the best single feature reaches only 0.85 after the fix. Thresholds are chosen on validation only, every comparison uses 3 seeds, and a data fingerprint stops training or SHAP on mismatched data. Limitation: one random stratified split per dataset and no cross-dataset test yet."

? **Q10: Where does this model sit in an IoT network, and is it fast enough?**

"On the **IoT edge gateway**, not on sensors. On one CPU thread the FP16 model is 159 KB and takes about 6.5 ms per packet (around 1,660 packets/s). It raises explainable alerts; we have not yet tested it on a real Raspberry Pi, which is a next step."

? Q11: How does SHAP work in simple terms?

"SHAP comes from Shapley values in game theory. Each input feature is a 'player', and SHAP measures how much each one pushed this packet's score toward Attack or Normal, compared with an average background. We use the gradient-based version for neural networks."